

```

diff -ruN a/.github/workflows/build-ios.yml b/.github/workflows/build-ios.yml
--- a/.github/workflows/build-ios.yml      1970-01-01 00:00:00.000000000 +0000
+++ b/.github/workflows/build-ios.yml      2026-10-05 11:22:56.699897973 +0000
@@ -0,0 +1,39 @@
+name: Build iOS
+
+on:
+  workflow_dispatch:
+  push:
+    branches: [ios-port]
+
+jobs:
+  build:
+    name: iOS
+    runs-on: macos-latest
+    steps:
+      - uses: actions/checkout@v4
+
+      # The Windows build packages FFmpeg.exe, its DLLs and a demo video. None o
+      # that exists or runs on iOS, so leave it out of the iOS package only (the
+      # mod.json in the repo, and the Windows build, are not touched).
+      - name: Drop Windows-only resources from the iOS package
+        run: |
+          python3 - <<'PY'
+          import json
+          m = json.load(open('mod.json', encoding='utf-8'))
+          files = m.get('resources', {}).get('files', [])
+          drop = ('ffmpeg', '.dll', '.exe', '.mp4')
+          m['resources']['files'] = [f for f in files if not any(d in f.lower()
+          json.dump(m, open('mod.json', 'w', encoding='utf-8'), indent=2, ensure
+          print('resources:', len(files), '->', len(m['resources']['files']))
+          PY
+
+      - name: Build the mod
+        uses: geode-sdk/build-geode-mod@main
+        with:
+          target: iOS
+          sdk: v5.10.1
+
+      - uses: actions/upload-artifact@v4
+        with:
+          name: guccibot-ios
+          path: ./build/*.geode
diff -ruN a/CMakeLists.txt b/CMakeLists.txt

```

--- a/CMakeLists.txt 2026-10-05 11:22:56.403547930 +0000

+++ b/CMakeLists.txt 2026-10-05 11:22:56.674361625 +0000

@@ -5,14 +5,39 @@

```
project(GucciBot VERSION 2.0.0 LANGUAGES CXX)
```

```
+# iOS: arm64 and no FFmpeg / SafetyHook / Winsock. GucciBot's Windows build is
+# unchanged; everything iOS-specific is behind this one flag (see src/ios/).
```

```
+set(GB_IS_IOS FALSE)
```

```
+if (GEODE_TARGET_PLATFORM STREQUAL "ios" OR CMAKE_SYSTEM_NAME STREQUAL "ios")
```

```
+    set(GB_IS_IOS TRUE)
```

```
+endif()
```

```
+
```

```
    file(GLOB_RECURSE SOURCES src/*.cpp)
```

```
+if (GB_IS_IOS)
```

```
+    # Absense's pathfinder and the MCP server read GD's memory at raw Windows x6
```

```
+    # offsets / listen on a Winsock socket; the FFmpeg render pipeline needs
```

```
+    # FFmpeg. src/ios/ has inert stand-ins with the same names (see
```

```
+    # src/ios/absense_stub.hpp). Everything else is built as on Windows.
```

```
+    list(FILTER SOURCES EXCLUDE REGEX "/src/absense/")
```

```
+    list(FILTER SOURCES EXCLUDE REGEX "/src/mcp/")
```

```
+    list(FILTER SOURCES EXCLUDE REGEX "/src/render/(render|video_decoder|textu
```

```
+    file(GLOB_RECURSE GB_IOS_SOURCES src/ios/*.cpp)
```

```
+    list(APPEND SOURCES ${GB_IOS_SOURCES})
```

```
+    # absense/compat/bot.cpp is replaced by src/ios/absense_stub.cpp; the compat
```

```
+    # headers (settings types, Bot shim) are still used as they are.
```

```
+endif()
```

```
+
```

```
    add_library(${PROJECT_NAME} SHARED ${SOURCES})
```

```
-target_include_directories(${PROJECT_NAME} PRIVATE
```

```
-    src
```

```
-    third_party/ffmpeg/include
```

```
-)
```

```
+target_include_directories(${PROJECT_NAME} PRIVATE src)
```

```
+if (GB_IS_IOS)
```

```
+    # Ahead of src/, so "absense/pathfinder/pathfinder.hpp" and friends resolve
```

```
+    # to the inert shims instead of the real headers.
```

```
+    target_include_directories(${PROJECT_NAME} BEFORE PRIVATE src/ios/shims)
```

```
+else()
```

```
+    target_include_directories(${PROJECT_NAME} PRIVATE third_party/ffmpeg/includ
```

```
+endif()
```

```
if(NOT DEFINED ENV{GEODE_SDK})
```

```
    message(FATAL_ERROR "Unable to find Geode SDK! Please define GEODE_SDK envir
```

@@ -24,8 +49,10 @@

```

set(IMGUI_VERSION "v1.91.4" CACHE STRING "Pin imgui version" FORCE)
CPMAddPackage("gh:matcool/gd-imgui-cocos#97643336209b760a3d59d83b770546b1cfef4e7")
-CPMAddPackage("gh:cursey/safetyhook#main")
-CPMAddPackage("gh:zyantific/zydis#bfffbb61")
+if (NOT GB_IS_IOS)
+    CPMAddPackage("gh:cursey/safetyhook#main")    # x86/x64 only
+    CPMAddPackage("gh:zyantific/zydis#bfffbb61")  # x86/x64 only
+endif()
CPMAddPackage(
    NAME ZLIB
    GITHUB_REPOSITORY madler/zlib
@@ -42,18 +69,23 @@
# Geode's precompiled header force-includes Geode.hpp (and so windows.h, and
# so winsock 1.1) ahead of line 1 of every source file. Opting this one file
# out of the PCH is the only way its own include order gets to matter.
-set_source_files_properties(
-    ${CMAKE_CURRENT_SOURCE_DIR}/src/mcp/mcp_server.cpp
-    PROPERTIES SKIP_PRECOMPILE_HEADERS ON
-)
+if (NOT GB_IS_IOS)
+    set_source_files_properties(
+        ${CMAKE_CURRENT_SOURCE_DIR}/src/mcp/mcp_server.cpp
+        PROPERTIES SKIP_PRECOMPILE_HEADERS ON
+    )
+endif()

target_link_libraries(${PROJECT_NAME}
    imgui-cocos
-    safetyhook
    zlibstatic
    libGDR
-    ws2_32
)
+if (GB_IS_IOS)
+    target_compile_definitions(${PROJECT_NAME} PRIVATE GB_NO_FFMPEG GB_IOS)
+else()
+    target_link_libraries(${PROJECT_NAME} safetyhook ws2_32)
+endif()

target_compile_definitions(${PROJECT_NAME} PRIVATE
    MOD_VERSION="2.0.0-beta.2"
diff -ruN a/IOS_PORT.md b/IOS_PORT.md
--- a/IOS_PORT.md      1970-01-01 00:00:00.000000000 +0000
+++ b/IOS_PORT.md      2026-10-05 11:22:56.699909266 +0000
@@ -0,0 +1,41 @@
+# GucciBot on iOS (port notes)

```

```

+
+Target: Geometry Dash 2.208, iOS (arm64), Geode 5.10.1 (iOS Launcher). The Windo
+build is unchanged; everything iOS-specific is behind `GEODE_IS_WINDOWS` /
+`GB_IS_IOS` / `GB_NO_FFMPEG`.
+
+### Build
+
+GitHub Actions, workflow **Build iOS** (`.github/workflows/build-ios.yml`, macOS
+Actions tab -> Build iOS -> Run workflow -> download artifact `guccibot-ios` (th
+The workflow removes ffmpeg/.dll/.exe/.mp4 from the *iOS package's* resource lis
+
+Offline check (no SDK needed): `python3 tools/ios_audit.py .` -- builds the iOS
+list, follows the include graph as the iOS build sees it and fails on any header
+Windows-only construct that must not be reached. It does not replace the real co
+
+### What is real on iOS
+
+Engine, GBR6/BRR formats (little-endian, fixed-width -- identical on arm64), mac
+recording/playback, GUI (ImGui; a touch **G** button opens the menu, plus Pause/
+in a level), Calculate, autoclicker, hitboxes, trainers, click sounds, trailbuf,
+
+### What is replaced or off
+
+| Windows mechanism | iOS |
+|---|---|
+| 5 SafetyHook midhooks in GJBaseGameLayer::update (physics dt / step count / ea
+| 4 binary patches | #1 (hold buttons across reset) -> `resetLevelVariables` hoo
+| raw `CCActionManager::update` hook | `$modify(CCActionManager)` |
+| shake / teleport RNG, checkpoint placement, y-velocity midhooks, queueCheckpoi
+| GD's RNG seed at `base+0x6c2e90` | `gucci::rngState()` -> dummy until `-DGB_IO
+| Absense pathfinder (`src/absense`) | inert stubs (`src/ios/`), Pathfinder tab
+| MCP server (Winsock) | inert stub |
+| FFMpeg renderer / video decoder | `render/renderer_stub.hpp` -- no rendering,
+| QueryPerformanceCounter (SCBF live clock) | `steady_clock` |
+| CCEGLView/GLFW hook | desktop only |
+
+### Finding the RNG offset
+Open GD 2.208 iOS in Ghidra/IDA, search the constants `0x343FD` and `0x269EC3`
+(`seed = seed * 214013 + 2531011`). The seed variable's offset from the image ba
+`GB_IOS_RNG_OFFSET`.
diff -ruN a/mod.json b/mod.json
--- a/mod.json 2026-10-05 11:22:56.404087989 +0000
+++ b/mod.json 2026-10-05 11:22:56.674636355 +0000
@@ -2,7 +2,8 @@
    "geode": "5.10.1",
    "version": "v2.0.0-beta.2",
    "gd": {

```

```

-     "win": "2.2081"
+     "win": "2.2081",
+     "ios": "2.208"
    },
    "id": "nigelx1.guccibot",
    "name": "GucciBot",
diff -ruN a/src/audio/bigbrrr.cpp b/src/audio/bigbrrr.cpp
--- a/src/audio/bigbrrr.cpp      2026-10-05 11:22:56.407950776 +0000
+++ b/src/audio/bigbrrr.cpp      2026-10-05 11:22:56.676922855 +0000
@@ -1,3 +1,4 @@
+#include "platform.hpp"
+    #include "audio/bigbrrr.hpp"
+    #include "audio/gameaudiomute.hpp"
+    #include "gui/gui.hpp"
diff -ruN a/src/core/GucciBot.hpp b/src/core/GucciBot.hpp
--- a/src/core/GucciBot.hpp      2026-10-05 11:22:56.408129338 +0000
+++ b/src/core/GucciBot.hpp      2026-10-05 11:22:56.676962583 +0000
@@ -1318,3 +1318,9 @@
    }

    } // namespace gucci
+
+// Per-physics-step logic shared by the Windows midhooks and the iOS step loop
+// (engine_updater.cpp, hooks/hook_gjbasegamelayer.cpp). No-ops unless the engine
+// is enabled.
+void gucciEarlyUpdateStep();
+void gucciFrameUpdateStep();
diff -ruN a/src/core/engine_core.cpp b/src/core/engine_core.cpp
--- a/src/core/engine_core.cpp    2026-10-05 11:22:56.408290199 +0000
+++ b/src/core/engine_core.cpp    2026-10-05 11:22:56.677072847 +0000
@@ -1,3 +1,4 @@
+#include "platform.hpp"
+    #include "core/GucciBot.hpp"
+    #include "analysis/ac/framewindow.hpp"
+    #include "gui/gui.hpp"
@@ -83,7 +84,7 @@
    // Same GD fast-rand global updateRandomSeedOnReset() rewinds on
    // every reset -- see the comment on SavedCheckpointState::m_rngState
    // for why this needs to be captured per-checkpoint now.
-    state.m_rngState = *reinterpret_cast<uint64_t*>(geode::base::get() + 0x6
+    state.m_rngState = gucci::rngState();
    // Every other random source a replay depends on, captured with it.
    state.m_teleportRandomState = GucciEngine::get()->replay.m_teleportRandomState;
    state.m_advRandStates.clear();
@@ -270,7 +271,7 @@
    // already rewound this to the attempt-start value earlier in the
    // same resetLevel() call -- restoring the checkpoint's own captured

```

```

        // value here is what actually fixes the per-checkpoint RNG gap.
-    *reinterpret_cast<uint64_t*>(geode::base::get() + 0x6c2e90) = state.m_rn
+    gucci::rngState() = state.m_rngState;
    GucciEngine::get()->replay.m_teleportRandomState = state.m_teleportRando
    for (size_t i = 0;
        i < state.m_advRandStates.size() && i < m_advancedRandom.size(); i+
@@ -2453,8 +2454,15 @@
    log::info("[GucciBot] =====");
    log::info("[GucciBot] " MOD_VERSION " initialized - {} macros", storedMa

+#ifndef GEODE_IS_WINDOWS
    gbcheck::run(5, 4, GBR6_VERSION, BRR_FORMAT_VERSION, MOD_VERSION,
        &editcoretest::run);
+#else
+    // iOS installs no raw midhooks or binary patches (see src/ios/), so the
+    // nothing to count; the format and save-dir checks still run.
+    gbcheck::run(0, 0, GBR6_VERSION, BRR_FORMAT_VERSION, MOD_VERSION,
+        &editcoretest::run);
+#endif
    }

    static PauseLayer* findOpenPauseLayerRecursive(CCNode* node) {
diff -ruN a/src/core/engine_updater.cpp b/src/core/engine_updater.cpp
--- a/src/core/engine_updater.cpp      2026-10-05 11:22:56.408348511 +0000
+++ b/src/core/engine_updater.cpp      2026-10-05 11:22:56.677118467 +0000
@@ -21,7 +21,9 @@
    #include <Geode/Geode.hpp>
    #include <Geode/modify/CCScheduler.hpp>
    #include <Geode/modify/CCDirector.hpp>
+#ifndef GEODE_IS_WINDOWS
    #include <safetyhook.hpp>
+#endif
    #include <fstream>
    #include <chrono>
    #include <cmath>
@@ -575,6 +577,7 @@
    // unconditionally once installed, completely independent of the
    // GB7CCScheduler/GB7CCDirector path below that already respected
    // `enabled` correctly. Added the same guard to all six.
+#ifndef GEODE_IS_WINDOWS
    static void physDtMidhook(SafetyHookContext& ctx) {
        auto* pl = GJBaseGameLayer::get();
        if (!pl)
@@ -588,7 +591,10 @@
        ctx.xmm1.f64[0] *= upd.m_tps / 60.0;
        ctx.rip += 0x08;
    }

```

```

+ #endif

+
+ #ifdef GEODE_IS_WINDOWS
+     static void physStepCountMidhook(SafetyHookContext& ctx) {
+         auto* gb = GucciEngine::get();
+         if (!gb->enabled)
+@@ -604,7 +610,10 @@
+             return;
+             ctx.rdx = 2 - upd.estimatedStepCount;
+     }
+ #endif
+
+
+ #ifdef GEODE_IS_WINDOWS
+     static void restorePhysDtMidhook(SafetyHookContext& ctx) {
+         auto* gb = GucciEngine::get();
+         if (!gb->enabled)
+@@ -620,8 +629,10 @@
+             return;
+             ctx.xmm9.f64[0] = upd.getPhysicsDt() * upd.estimatedStepCount;
+     }
+ #endif
+
+ -static void earlyUpdateMidhook(SafetyHookContext&) {
+ +
+ +void gucciEarlyUpdateStep() {
+     auto* gb = GucciEngine::get();
+     if (!gb->enabled)
+         return;
+@@ -707,7 +718,7 @@
+     }
+ }
+
+ -static void frameUpdateMidhook(SafetyHookContext&) {
+ +void gucciFrameUpdateStep() {
+     auto* gb = GucciEngine::get();
+     if (!gb->enabled)
+         return;
+@@ -1174,6 +1185,7 @@
+     }
+ };
+
+ #ifdef GEODE_IS_WINDOWS
+     constexpr int ACTIONMGR_UPDATE_OFFSET = 0x38B90;
+     static void (*actionMgrOrig)(void*, float) = nullptr;

```

```

@@ -1195,8 +1207,10 @@
    util_midhook(geode::base::get() + 0x237A7C, "physDt", physDtMidhook);
    util_midhook(geode::base::get() + 0x237DCE, "physStepCount", physStepCountMi
    util_midhook(geode::base::get() + 0x238F6E, "restorePhysDt", restorePhysDtMi
-   util_midhook(geode::base::get() + 0x237E42, "earlyUpdate", earlyUpdateMidhoo
-   util_midhook(geode::base::get() + 0x238BAA, "frameUpdate", frameUpdateMidhoo
+   util_midhook(geode::base::get() + 0x237E42, "earlyUpdate",
+               [](SafetyHookContext&) { gucciEarlyUpdateStep(); });
+   util_midhook(geode::base::get() + 0x238BAA, "frameUpdate",
+               [](SafetyHookContext&) { gucciFrameUpdateStep(); });

    actionMgrOrig =
        reinterpret_cast<void (*) (void*, float)>(geode::base::getCocos() + ACTIO
@@ -1244,3 +1258,18 @@
    if (g_patchFailures)
        geode::log::error("[GucciBot] {} of 4 binary patches FAILED to apply", g
    }
+ #endif // GEODE_IS_WINDOWS (raw x64 hooks/patches)
+
+ #ifndef GEODE_IS_WINDOWS
+ #include <Geode/modify/CCActionManager.hpp>
+ // Windows replaces CCActionManager::update with a raw hook (actionMgrHook above
+ // iOS hooks it by name.
+ class $modify(GB7ActionManager, CCActionManager) {
+     void update(float dt) {
+         auto* gb = GucciEngine::get();
+         if (gb->enabled && gb->updater.m_onlyRefresh)
+             return;
+         CCActionManager::update(dt);
+     }
+ };
+ #endif
diff -ruN a/src/gui/gui.cpp b/src/gui/gui.cpp
--- a/src/gui/gui.cpp    2026-10-05 11:22:56.408571614 +0000
+++ b/src/gui/gui.cpp    2026-10-05 11:22:56.677273602 +0000
@@ -1,3 +1,4 @@
+ #include "platform.hpp"
+ #include "gui/gui.hpp"

    #include "analysis/trajectory.hpp"
@@ -10640,7 +10641,7 @@
        addLine(fmt::format("Time warp: {:.2f}x", pl->m_gameState.m_timeWarp
        if (h.showRandom) {
            // The GD fast-rand global updateRandomSeedOnReset rewinds.
-            uint64_t const fastRand = *reinterpret_cast<uint64_t*>(geode::base::
+            uint64_t const fastRand = gucci::rngState();
+            addLine(fmt::format("Random state: {}", fastRand));

```

```

        addLine(fmt::format("Shake state: {} ", engine->replay.m_shakeRandomS
        addLine(fmt::format("Teleport state: {} ", engine->replay.m_teleportR
@@ -10780,6 +10781,62 @@
        ImGui::End();
    }

+ #ifndef GEODE_IS_WINDOWS
+     // Touch-only controls for phones. The menu and frame advance are opened by
+     // keyboard hotkeys (hacks/keybinds.cpp), which an iPhone does not have --
+     // without this the menu could never be opened at all.
+     static void displayTouchControls() {
+         auto* ui = MenuInterface::get();
+         auto* engine = GucciEngine::get();
+         if (!ui || !ui->setupComplete || !engine)
+             return;
+         if (ui->shown)
+             return; // the open menu has its own close button
+
+         auto* vp = ImGui::GetMainViewport();
+         float const sz = ImGui::GetFontSize() * 2.6f;
+         float const margin = vp->Size.x * 0.06f; // clear of the notch / safe ar
+
+         ImGui::SetNextWindowPos(ImVec2(vp->Pos.x + margin, vp->Pos.y + sz * 0.4f
+         ImGui::SetNextWindowSize(ImVec2(0, 0), ImGuiCond_Always);
+         ImGui::SetNextWindowBgAlpha(0.30f);
+         ImGui::Begin("##gbutton",
+             nullptr,
+             ImGuiWindowFlags_NoDecoration | ImGuiWindowFlags_AlwaysAuto
+                 ImGuiWindowFlags_NoSavedSettings | ImGuiWindowFlags_NoF
+                 ImGuiWindowFlags_NoNav | ImGuiWindowFlags_NoMove);
+         if (ui->fontBody)
+             ImGui::PushFont(ui->fontBody);
+
+         if (ImGui::Button("G", ImVec2(sz, sz))) {
+             // Same as the keyboard menu hotkey, minus the keyboard.
+             if (engine->ttrEnabledConflict) {
+                 GucciEngine::showTtrEnabledNotification();
+             } else {
+                 ui->shown = true;
+                 ui->anim.opening = true;
+                 ui->anim.closing = false;
+                 ui->anim.openProgress = 0.f;
+             }
+         }
+
+         if (PlayLayer::get()) {
+             ImGui::SameLine();

```

```
+         if (ImGui::Button(engine->updater.m_paused ? "Run" : "Pause", ImVec2
+             engine->updater.togglePaused());
+         if (engine->updater.m_paused) {
+             ImGui::SameLine();
+             if (ImGui::Button("Step", ImVec2(sz * 1.6f, sz)))
+                 engine->updater.m_stepOnce_ = true;
+         }
+     }
+
+     if (ui->fontBody)
+         ImGui::PopFont();
+     ImGui::End();
+ }
+#endif
+
+ $on_mod(Loaded) {
+     ImGuiCocos::get()
+         .setup([] {
@@ -10819,6 +10876,9 @@
+             displayCalculatingHUD();
+             displayGameplayHUD();
+             displayAccuracyHUD();
+
+#ifndef GEODE_IS_WINDOWS
+             displayTouchControls();
+#endif
+
+         });
+     }
diff -ruN a/src/hooks/hook_cceglview.cpp b/src/hooks/hook_cceglview.cpp
--- a/src/hooks/hook_cceglview.cpp      2026-10-05 11:22:56.408999713 +0000
+++ b/src/hooks/hook_cceglview.cpp      2026-10-05 11:22:56.677541496 +0000
@@ -1,3 +1,4 @@
+#ifdef GEODE_IS_WINDOWS // GLFW window callbacks and desktop render resizing onl
+    #include <Geode/Geode.hpp>
+    #include <Geode/modify/CCEGLView.hpp>
@@ -37,3 +38,4 @@
+        CCEGLView::onGLFWWindowSizeFunCallback(window, width, height);
+    }
+};
+#endif // GEODE_IS_WINDOWS
diff -ruN a/src/hooks/hook_gjbasegamelayer.cpp b/src/hooks/hook_gjbasegamelayer.c
--- a/src/hooks/hook_gjbasegamelayer.cpp      2026-10-05 11:22:56.409027182 +00
+++ b/src/hooks/hook_gjbasegamelayer.cpp      2026-10-05 11:22:56.677554056 +00
@@ -1,3 +1,4 @@
+#include "platform.hpp"
+
+#include "core/GucciBot.hpp"
```

```

#include "analysis/trajectory.hpp"
#include "audio/playsound.hpp"
@@ -6,7 +7,9 @@
#include "trainers/jupiterghost.hpp"
#include "trainers/trainerghost.hpp"
#include "render/renderer.hpp"
+#ifndef GEODE_IS_WINDOWS
#include <safetyhook.hpp>
+#endif

#include <Geode/Geode.hpp>
#include <Geode/modify/GJBaseGameLayer.hpp>
@@ -33,6 +36,7 @@
    void log(std::string const& line);
}

+#ifndef GEODE_IS_WINDOWS
static void shakeRandomOverride(SafetyHookContext& ctx) {
    uint64_t& state = GucciEngine::get()->replay.m_shakeRandomState;
    state = (int)((214013 * state + 2531011) >> 16) & 0x7FFF;
@@ -43,18 +47,24 @@
    // side effect.
    ctx.rip += 6;
}
+#endif
+

// ENGINE_AUDIT $1.4. The teleport trigger's random pick, from the macro's
// seeded teleport state instead of GD's rand(). m_teleportRandomState was
// stored and restored with checkpoints and reseeded every attempt, but the
// hook that reads it was never installed. This one sits after the call (no
// rip skip) -- Silicate's placement.
+#ifndef GEODE_IS_WINDOWS
static void teleportRandomOverride(SafetyHookContext& ctx) {
    uint64_t& state = GucciEngine::get()->replay.m_teleportRandomState;
    state = (int)((214013 * state + 2531011) >> 16) & 0x7FFF;
    ctx.rax = (uintptr_t)state;
}
+#endif
+

+#ifndef GEODE_IS_WINDOWS
static void overrideCheckpointPlacement(SafetyHookContext& ctx) {
    ctx.rip += 5;
    auto* pl = PlayLayer::get();
@@ -68,6 +78,8 @@
    return;
}

```

```

        pl->queueCheckpoint();
    }
+ #endif
+
    class $modify(GB7GJBaseGameLayer, GJBaseGameLayer) {
        struct Fields {
@@ -207,6 +219,80 @@
            CCLayer::update(dt);
        }

+ #ifndef GEODE_IS_WINDOWS
+     // Windows NOPs a block of GJBaseGameLayer::resetLevelVariables so held butt
+     // survive a reset (binary patch #1). iOS has no raw patch: snapshot the hol
+     // state and put it back after the original has run.
+     void resetLevelVariables() {
+         auto* gb = GucciEngine::get();
+         if (!gb->enabled || !m_player1 || !m_player2) {
+             GJBaseGameLayer::resetLevelVariables();
+             return;
+         }
+         auto h1 = m_player1->m_holdingButtons;
+         auto h2 = m_player2->m_holdingButtons;
+         bool const l1 = m_player1->m_holdingLeft, r1 = m_player1->m_holdingRight
+         bool const l2 = m_player2->m_holdingLeft, r2 = m_player2->m_holdingRight
+         GJBaseGameLayer::resetLevelVariables();
+         m_player1->m_holdingButtons = h1;
+         m_player2->m_holdingButtons = h2;
+         m_player1->m_holdingLeft = l1;
+         m_player1->m_holdingRight = r1;
+         m_player2->m_holdingLeft = l2;
+         m_player2->m_holdingRight = r2;
+     }
+ #endif
+
+     // One call == one drawn frame's worth of physics.
+     //
+     // Windows: GD runs its own step loop and the midhooks (engine_updater.cpp)
+     // set the step count and the step length and call the per-step hooks from
+     // inside it, so this is just GD's update.
+     //
+     // iOS has no raw hooks, so the steps are run from here: one GD update per
+     // step, handed exactly one physics step's delta (getModifiedDelta below
+     // returns it), with the per-step hooks around it. Exact at the game's nativ
+     // 240 TPS; other rates depend on how GD turns that delta into a step count,
+     // which is why the first calls log a measured ratio (see the [ios-step] lin
+     void runPhysicsSteps(float dt) {

```

```

+ #ifdef GEODE_IS_WINDOWS
+     GJBaseGameLayer::update(dt);
+ #else
+     auto& upd = GucciEngine::get()->updater;
+     bool const fastBypass =
+         upd.useFastLockDelta() || !upd.m_lockDelta || upd.m_analysisBatch >
+     int steps = upd.estimatedStepCount;
+     // Slow lock delta inside a level: runSlowLockDelta hands this update
+     // exactly one step, and the midhooks leave GD's own count alone then.
+     if (!fastBypass && PlayLayer::get())
+         steps = 1;
+     if (steps <= 0) {
+         // Editor playback preview runs on GD's own timing.
+         if (LevelEditorLayer::get() && LevelEditorLayer::get()->m_playbackAc
+             GJBaseGameLayer::update(dt);
+         return;
+     }
+     float const stepDt = (float)upd.getPhysicsDt();
+     static int s_diagCalls = 0;
+     for (int i = 0; i < steps; i++) {
+         gucciEarlyUpdateStep();
+         double const t0 = m_gameState.m_levelTime;
+         GJBaseGameLayer::update(stepDt);
+         gucciFrameUpdateStep();
+         // Self-check: the level clock should advance by exactly one physics
+         // step per call. A ratio far from 1.0 means the iOS step model is
+         // off at this TPS -- that log line is what needs sending back.
+         if (s_diagCalls < 40 || s_diagCalls % 2000 == 0) {
+             double const adv = m_gameState.m_levelTime - t0;
+             geode::log::info(
+                 "[GucciBot][ios-step] tps={} stepDt={:.6f} levelTimeAdv={:.6
+                 upd.m_tps, stepDt, adv, stepDt > 0 ? adv / stepDt : 0.0, ste
+             }
+             ++s_diagCalls;
+         }
+     }
+ #endif
+ }
+
+ void update(float dt) {
+     auto* gb = GucciEngine::get();
+     auto& upd = gb->updater;
@@ -248,7 +334,7 @@
+         extrapolateVisualUpdates(dt);
+     } else {
+         loadActualState();
-         GJBaseGameLayer::update(dt);
+         runPhysicsSteps(dt);

```

```

        storeActualState();
    }
} else {
@@ -263,7 +349,7 @@
    auto* lel = LevelEditorLayer::get();
    bool editorPlayback = lel && lel->m_playbackActive;
    if (lockDeltaActive || upd.estimatedStepCount != 0 || editorPlayback
-        GJBaseGameLayer::update(dt);
+        runPhysicsSteps(dt);
        storeActualState();
    }
}
@@ -430,7 +516,7 @@
    if (action.m_type == gb::ActionType::Death) {
        upd.m_expectsDeath = true;
        gb->replay.m_startingSeedThisAttempt =
-            *reinterpret_cast<uint64_t*>(geode::base::get() + 0x6c2e90);
+            gucci::rngState();
        return;
    }
    if (action.m_type == gb::ActionType::RestartFull) {
@@ -907,6 +993,7 @@
    }
};

#ifdef GEODE_IS_WINDOWS
$execute {
    util_midhook(geode::base::get() + 0x23E173, "shakeRandom1", shakeRandomOverr
    util_midhook(geode::base::get() + 0x23E1A1, "shakeRandom2", shakeRandomOverr
@@ -915,3 +1002,4 @@
    util_midhook(geode::base::get() + 0x20FEDC, "teleportRandomOverride", telepo
    util_midhook(geode::base::get() + 0x3A3657, "checkpointPlacement", overrideC
}
#endif // GEODE_IS_WINDOWS (raw midhooks)
diff -ruN a/src/hooks/hook_playerobject.cpp b/src/hooks/hook_playerobject.cpp
--- a/src/hooks/hook_playerobject.cpp    2026-10-05 11:22:56.409094726 +0000
+++ b/src/hooks/hook_playerobject.cpp    2026-10-05 11:22:56.677606606 +0000
@@ -43,6 +43,7 @@
    // that with this midhook -- quantise the value in xmm1 to the finer step and
    // jump past GD's own 0.001 rounding. Until 2026-09-27 GucciBot had only the
    // setter half, so High TPS Precision did half of what it says.
#ifdef GEODE_IS_WINDOWS
static void yVelocityRoundMidhook(SafetyHookContext& ctx) {
    double const quantum = yVelocityQuantum();
    if (quantum >= kVanillaQuantum)
@@ -54,6 +55,7 @@
$execute {

```

```

        util_midhook(geode::base::get() + 0x38c315, "yVelocityRound", yVelocityRound
    }
#endif // GEODE_IS_WINDOWS (inline y-velocity rounding midhook)

// Congregation slope diagnostic, see engine_updater.cpp.
namespace gucci::slopediag {
diff -ruN a/src/hooks/hook_playlayer.cpp b/src/hooks/hook_playlayer.cpp
--- a/src/hooks/hook_playlayer.cpp      2026-10-05 11:22:56.409119561 +0000
+++ b/src/hooks/hook_playlayer.cpp      2026-10-05 11:22:56.677622500 +0000
@@ -1,3 +1,4 @@
#include "platform.hpp"
#include "core/GucciBot.hpp"
#include "render/renderer.hpp"
#include "hacks/autoclicker.hpp"
@@ -18,7 +19,9 @@

#include <Geode/Geode.hpp>
#include <Geode/modify/PlayLayer.hpp>
#ifdef GEODE_IS_WINDOWS
#include <safetyhook.hpp>
#endif
#include <fmt/format.h>

using namespace geode::prelude;
@@ -295,7 +298,7 @@
    void updateRandomSeedOnReset() {
        auto* gb = GucciEngine::get();
        auto& rs = gb->replay;
-        uint64_t& state = *reinterpret_cast<uint64_t*>(geode::base::get() + 0x6c
+        uint64_t& state = gucci::rngState();

        if (!gb->updater.m_expectsDeath)
            rs.m_startingSeedThisAttempt = rs.m_startingSeed;
@@ -882,6 +885,7 @@
    }
};

#ifdef GEODE_IS_WINDOWS
// GD's checkpoint key/button handler (2.2081 Windows): if practice mode is on,
// it sets m_tryPlaceCheckpoint and GD places the checkpoint later, part-way
// through a tick. Silicate replaces it so the checkpoint is placed in the
@@ -919,3 +923,4 @@
        "PlayLayer::queueCheckpoint",
        tulip::hook::TulipConvention::Default);
}
#endif // GEODE_IS_WINDOWS (raw queueCheckpoint hook)
diff -ruN a/src/hooks/util_midhook.hpp b/src/hooks/util_midhook.hpp

```

```

--- a/src/hooks/util_midhook.hpp      2026-10-05 11:22:56.409195303 +0000
+++ b/src/hooks/util_midhook.hpp      2026-10-05 11:22:56.677672036 +0000
@@ -1,19 +1,23 @@
    #pragma once

    #include <Geode/Geode.hpp>
-#include <safetyhook.hpp>
    #include <string>
    #include <unordered_map>

-namespace gucci {
+#ifdef GEODE_IS_WINDOWS
+#include <safetyhook.hpp>
+#endif

-    static std::unordered_map<std::string, safetyhook::MidHook> g_midHooks;
+namespace gucci {

    inline int g_midhookAttempts = 0;
    inline int g_midhookFailures = 0;
    inline int g_patchAttempts = 0;
    inline int g_patchFailures = 0;

+#ifdef GEODE_IS_WINDOWS
+    static std::unordered_map<std::string, safetyhook::MidHook> g_midHooks;
+
    inline bool util_midhook(uintptr_t address, const std::string& name, safetyh
        ++g_midhookAttempts;
        auto hook = safetyhook::create_mid(reinterpret_cast<void*>(address), fn)
@@ -26,5 +30,6 @@
        geode::log::info("[GucciBot] Installed midhook '{}'", name);
        return true;
    }
+#endif

    } // namespace gucci
diff -ruN a/src/ios/absense_stub.cpp b/src/ios/absense_stub.cpp
--- a/src/ios/absense_stub.cpp 1970-01-01 00:00:00.000000000 +0000
+++ b/src/ios/absense_stub.cpp 2026-10-05 11:22:56.678664656 +0000
@@ -0,0 +1,64 @@
+// iOS: the parts of absense/compat/bot.cpp the rest of GucciBot links against,
+// answering "no pathfinder". See ios/absense_stub.hpp.
+
+#include "absense/compat/bot.hpp"
+#include "absense/glue.hpp"
+
+TrajectoryManager& Bot::trajectory() {

```

```

+     static TrajectoryManager inst;
+     return inst;
+}
+
+AbsensePathfinder& Bot::pathfinder() {
+     static AbsensePathfinder inst;
+     return inst;
+}
+
+AbsAutoclicker& Bot::autoclicker() {
+     static AbsAutoclicker inst;
+     return inst;
+}
+
+// The Pathfinder tab always uses GucciBot's own (Classic) engine here.
+bool absense::classicSelected() {
+     return true;
+}
+
+void absense::selectClassic(bool) {}
+
+bool absense::startPathfinder() {
+     return false;
+}
+
+bool absense::isRunning() {
+     return false;
+}
+
+bool absense::requestStart(bool) {
+     return false;
+}
+
+bool absense::startPending() {
+     return false;
+}
+
+void absense::servicePendingStart() {}
+
+bool absense::startFromBeginning() {
+     return false;
+}
+
+void absense::setStartFromBeginning(bool) {}
+
+void absense::stopPathfinder() {}
+
+
+absense::Status absense::status() {
+     Status st;
+     st.message = "Absense's pathfinder is not available on iOS.";
+     return st;
+}
+
+
+matjson::Value absense::simulate(std::vector<ScriptTick> const&, int, int) {
+     return matjson::Value::object();
+}

```

```

+}
+matjson::Value absense::simVsReal(std::vector<ScriptTick> const&, int, int) {
+    return matjson::Value::object();
+}
+
+
+void absense::distrust(int) {}
+void absense::clearDistrust() {}
+std::vector<int> absense::distrustedList() {
+    return {};
+}
diff -ruN a/src/ios/absense_stub.hpp b/src/ios/absense_stub.hpp
--- a/src/ios/absense_stub.hpp 1970-01-01 00:00:00.000000000 +0000
+++ b/src/ios/absense_stub.hpp 2026-10-05 11:22:56.678655405 +0000
@@ -0,0 +1,64 @@
+#pragma once
+// iOS: Absense's pathfinder is not built. It reads and writes GD's memory at ra
+// Windows x64 struct offsets (absense/world/offsets.hpp), which are wrong on
+// arm64, and it checks them against the Windows binary at level load. These are
+// inert stand-ins with the same names and signatures, so the hooks, the updater
+// and the GUI compile unchanged. Reached through src/ios/shims/absense/*.hpp,
+// which only the iOS build puts ahead of the real headers on the include path.
+//
+// Every query answers "nothing is running", so none of the hooks that consult
+// them change behaviour. The Pathfinder tab uses GucciBot's own Classic engine.
+
+#include <Geode/Geode.hpp>
+
+class TrajectoryManager {
+public:
+    bool exists() const { return false; }
+    void init() {}
+    void uninit() {}
+    bool isFakePlayer(PlayerObject*) { return false; }
+    void noteKiller(PlayerObject*, GameObject*) {}
+    void hasDied(PlayerObject*) {}
+    void setDelta(float) {}
+    bool drawing() const { return false; }
+    bool simulating() const { return false; }
+    void simBreak(GameObject*) {}
+    void realStateChanged() {}
+};
+
+class AbsensePathfinder {
+public:
+    enum class Phase { Idle, Deciding, Committing, Backtracking, Probing, Restor
+
+    bool isRunning() const { return false; }

```

```

+   Phase phase() const { return Phase::Idle; }
+   bool drivesGame() const { return false; }
+   bool start() { return false; }
+   void tick() {}
+   void afterLiveTick() {}
+   void liveInput() {}
+   bool onWouldDie(PlayerObject*, GameObject*) { return false; }
+   void onLevelComplete() {}
+   void onForeignReset() {}
+   void onQuit() {}
+   void noteFrameCost(double, double) {}
+};
+
+namespace world {
+   class World {
+   public:
+       static void afterRealTick(GJBaseGameLayer*, int) {}
+       static void turnOff(const char*) {}
+   };
+
+   inline void onEvent(int, int, int) {}
+} // namespace world
+
+namespace absense::judge {
+   inline bool active() { return false; }
+   inline void noteDeath(GameObject*) {}
+   inline void noteComplete() {}
+   inline void distrust(int) {}
+   inline void clearDistrust() {}
+   inline bool distrusted(int) { return false; }
+} // namespace absense::judge
diff -ruN a/src/ios/mcp_stub.cpp b/src/ios/mcp_stub.cpp
--- a/src/ios/mcp_stub.cpp      1970-01-01 00:00:00.000000000 +0000
+++ b/src/ios/mcp_stub.cpp      2026-10-05 11:22:56.678674642 +0000
@@ -0,0 +1,20 @@
+// iOS: Assistant Access (the in-game MCP server) listens on a Winsock socket
+// and is meant for a desktop session talking to a PC. Inert here: start()
+// fails, so the Settings toggle switches itself back off.
+
+#include "mcp/mcp_server.hpp"
+
+namespace gucci::mcp {
+
+   bool Server::start(int) {
+       return false;
+   }
+
+   void Server::stop() {}

```

```

+ void Server::pump() {}
+ void Server::addTool(Tool t) {
+     m_tools.push_back(std::move(t));
+ }
+ void mcpFileLog(std::string const&) {}
+ void registerTools(Server&) {}
+
+} // namespace gucci::mcp
diff -ruN a/src/ios/shims/absense/judge.hpp b/src/ios/shims/absense/judge.hpp
--- a/src/ios/shims/absense/judge.hpp 1970-01-01 00:00:00.000000000 +0000
+++ b/src/ios/shims/absense/judge.hpp 2026-10-05 11:22:56.678645950 +0000
@@ -0,0 +1,3 @@
+#pragma once
+// iOS shim: see src/ios/absense_stub.hpp
+#include "ios/absense_stub.hpp"
diff -ruN a/src/ios/shims/absense/pathfinder/pathfinder.hpp b/src/ios/shims/absen
--- a/src/ios/shims/absense/pathfinder/pathfinder.hpp 1970-01-01 00:00:00.00000
+++ b/src/ios/shims/absense/pathfinder/pathfinder.hpp 2026-10-05 11:22:56.67859
@@ -0,0 +1,3 @@
+#pragma once
+// iOS shim: see src/ios/absense_stub.hpp
+#include "ios/absense_stub.hpp"
diff -ruN a/src/ios/shims/absense/trajectory/trajectory.hpp b/src/ios/shims/absen
--- a/src/ios/shims/absense/trajectory/trajectory.hpp 1970-01-01 00:00:00.00000
+++ b/src/ios/shims/absense/trajectory/trajectory.hpp 2026-10-05 11:22:56.67861
@@ -0,0 +1,3 @@
+#pragma once
+// iOS shim: see src/ios/absense_stub.hpp
+#include "ios/absense_stub.hpp"
diff -ruN a/src/ios/shims/absense/world/world.hpp b/src/ios/shims/absense/world/w
--- a/src/ios/shims/absense/world/world.hpp 1970-01-01 00:00:00.000000000 +00
+++ b/src/ios/shims/absense/world/world.hpp 2026-10-05 11:22:56.678634602 +00
@@ -0,0 +1,3 @@
+#pragma once
+// iOS shim: see src/ios/absense_stub.hpp
+#include "ios/absense_stub.hpp"
diff -ruN a/src/platform.hpp b/src/platform.hpp
--- a/src/platform.hpp 1970-01-01 00:00:00.000000000 +0000
+++ b/src/platform.hpp 2026-10-05 11:22:56.678711224 +0000
@@ -0,0 +1,45 @@
+#pragma once
+// Cross-platform helpers for the iOS port. Everything here is a no-op or a thin
+// wrapper on Windows, where GucciBot's behaviour is unchanged.
+
+#include <Geode/Geode.hpp>
+
+#include <chrono>

```

```

#include <cstdint>
#include <cstdint>
#include <cstring>
+
+// MSVC-only "secure" CRT helper; Apple clang does not provide it.
#ifndef GEODE_IS_WINDOWS
template <size_t N>
inline void strcpy_s(char (&dst)[N], const char* src) {
+   std::strncpy(dst, src, N - 1);
+   dst[N - 1] = '\0';
+}
#endif
+
namespace gucci {
+
+   // GD's fast_rand seed (the LCG 214013*x+2531011). On Windows it lives at a
+   // fixed address. Its address in the iOS binary is not known yet: find it in
+   // a disassembler (search for the constants 0x343FD and 0x269EC3), then
+   // build with -DGB_IOS_RNG_OFFSET=0x... (relative to geode::base::get()).
+   // 0 (the default) uses a private dummy, so on iOS macros are NOT
+   // deterministic across attempts that involve GD's random numbers.
#ifndef GB_IOS_RNG_OFFSET
#define GB_IOS_RNG_OFFSET 0
#endif
+   inline uint64_t& rngState() {
#ifdef GEODE_IS_WINDOWS
+       return *reinterpret_cast<uint64_t*>(geode::base::get() + 0x6c2e90);
#else
+       if constexpr (GB_IOS_RNG_OFFSET != 0) {
+           return *reinterpret_cast<uint64_t*>(geode::base::get() + GB_IOS_RNG_
+       } else {
+           static uint64_t s_dummyRng = 0;
+           return s_dummyRng;
+       }
#endif
+   }
+
+} // namespace gucci
diff -ruN a/src/render/dsp.cpp b/src/render/dsp.cpp
--- a/src/render/dsp.cpp      2026-10-05 11:22:56.409490324 +0000
+++ b/src/render/dsp.cpp      2026-10-05 11:22:56.677829822 +0000
@@ -1,3 +1,4 @@
#include "platform.hpp"
#include "dsp.hpp"

#include "renderer.hpp"
diff -ruN a/src/render/renderer.hpp b/src/render/renderer.hpp

```

```

--- a/src/render/rendererer.hpp    2026-10-05 11:22:56.409673629 +0000
+++ b/src/render/rendererer.hpp    2026-10-05 11:22:56.678030755 +0000
@@ -1,6 +1,10 @@
    #ifndef SL_RENDERERER_HPP
    #define SL_RENDERERER_HPP

+    #ifndef GB_NO_FFMPEG
+    #include "rendererer_stub.hpp"
+    #else
+
    #include <functional>
    #include "ffmpeg.hpp"
    #include "texture.hpp"
@@ -330,4 +334,6 @@

    } // namespace gucci

+    #endif // GB_NO_FFMPEG
+
    #endif
diff -ruN a/src/render/rendererer_stub.hpp b/src/render/rendererer_stub.hpp
--- a/src/render/rendererer_stub.hpp    1970-01-01 00:00:00.000000000 +0000
+++ b/src/render/rendererer_stub.hpp    2026-10-05 11:22:56.678108904 +0000
@@ -0,0 +1,116 @@
+    #pragma once
+    // iOS (GB_NO_FFMPEG): there is no FFmpeg export pipeline. Inert stand-ins for
+    // the parts of SLRenderer / VideoDecoder that the rest of the mod calls into.
+    // Names and signatures match render/rendererer.hpp and render/video_decoder.hpp.
+
+    #include "dsp.hpp"
+    #include "intro.hpp" // RenderIntroSettings: pure cocos2d, builds everywhere
+
+    #include <Geode/Geode.hpp>
+
+    #include <atomic>
+    #include <stdint>
+    #include <filesystem>
+    #include <string>
+    #include <vector>
+
+    namespace gucci {
+
+        struct SLRendererSettings {
+            int m_width = 1920;
+            int m_height = 1080;
+            uint32_t m_bitrate = 30'000'000;
+            std::string m_codec = "libx264";

```

```

+         int m_pixFmt = 0;
+
+         int m_fps = 60;
+
+         float m_afterEndTime = 3.0f;
+         double m_fadeInTime = 1.5;
+         double m_fadeOutTime = 1.5;
+         bool m_colorFix = true;
+         bool m_firstAttemptPause = false;
+
+         std::string m_outputPath = "output";
+         std::string m_extension = "mp4";
+         std::string m_audioCodec = "aac";
+         std::string m_renderArgs = "";
+
+         double m_musicVolume = 1.0;
+         double m_sfxVolume = 1.0;
+         double m_triggerSfxVolume = 1.0;
+         bool m_splitAudioTracks = false;
+         bool m_autoExitLevel = false;
+         bool m_previewAudio = false;
+
+         RenderIntroSettings m_intro;
+     };
+
+     struct SLRenderTexture {
+         void displayPreview() {}
+     };
+
+     class SLRenderer {
+     public:
+         static SLRenderer* get() {
+             static SLRenderer instance;
+             return &instance;
+         }
+
+         void queueStart() {}
+         void startIfQueued() { m_shouldStart = false; }
+         geode::Result<> writeAudio(std::vector<float>&, uint64_t, int) { return
+         void signalStop() {}
+         void flushAndStop() {}
+         void finishStop() {}
+         bool tryBeginFrame() { return true; }
+         void update(PlayLayer*) {}
+         bool tickIntro(PlayLayer*) { return false; }
+         bool introEnabled() const { return false; }
+         void displayPreview() { m_texture.displayPreview(); }

```

```

+     void setChannels(int c) { m_channels = c; }
+     void loadFFmpeg() {}
+     void loadSettingsFromGeode() {}
+
+     bool isRecording() const { return false; }
+     bool isFFmpegLoaded() const { return false; }
+     float getDt() const { return 1.f / (float)m_settings.m_fps; }
+     float getTime() const { return 0.f; }
+     int renderedFrames() const { return 0; }
+
+     struct LastRender {
+         bool pending = false;
+         bool success = false;
+         std::string path;
+         unsigned width = 0, height = 0, fps = 0;
+         double duration = 0.0;
+         uintmax_t fileSize = 0;
+     } m_lastRender;
+
+     SLRendererSettings m_settings;
+     bool m_shouldStart = false;
+     bool m_collectAudio = true;
+     bool m_shouldQuit = false;
+     bool m_startOnNextLevel = false;
+     bool m_autoVideoName = true;
+     std::string m_videoNameTemplate = "%name%_%rand%";
+     float m_fadeThreshold = 1.0f;
+     double m_time = 0;
+     SLRenderTexture m_texture;
+
+ private:
+     int m_channels = 2;
+ };
+
+ class VideoDecoder {
+ public:
+     bool open(const std::filesystem::path&) { return false; }
+     void close() {}
+     bool isOpen() const { return false; }
+     int width() const { return 0; }
+     int height() const { return 0; }
+     double durationSec() const { return 0.0; }
+     bool getFrameAt(double, std::vector<uint8_t>&) { return false; }
+ };
+
+ } // namespace gucci

```

diff -ruN a/src/render/video_decoder.hpp b/src/render/video_decoder.hpp

```

--- a/src/render/video_decoder.hpp      2026-10-05 11:22:56.409813181 +0000
+++ b/src/render/video_decoder.hpp      2026-10-05 11:22:56.678096935 +0000
@@ -1,11 +1,16 @@
    #pragma once

    #ifdef GB_NO_FFMPEG
    #include "render/render_stub.hpp"
    #else
    #include "render/ffmpeg.hpp"
    #endif

    #include <cstdint>
    #include <filesystem>
    #include <vector>

    #ifndef GB_NO_FFMPEG
    namespace gucci {

        // Decodes a local video file and serves individual frames by timestamp,
    @@ -108,3 +113,5 @@
        };

    } // namespace gucci
    #endif // GB_NO_FFMPEG
    +
    diff -ruN a/src/replay/scbf_input.cpp b/src/replay/scbf_input.cpp
    --- a/src/replay/scbf_input.cpp 2026-10-05 11:22:56.409855123 +0000
    +++ b/src/replay/scbf_input.cpp 2026-10-05 11:22:56.678133918 +0000
    @@ -5,7 +5,11 @@

    #include <Geode/Geode.hpp>

    #ifdef GEODE_IS_WINDOWS
    #include <Windows.h>
    #else
    #include <chrono>
    #endif

    #include <algorithm>
    #include <cmath>
    @@ -31,6 +35,7 @@
        } // namespace

        double clockSeconds() {
    #ifdef GEODE_IS_WINDOWS
        static double const freq = [] {
            LARGE_INTEGER f;

```

```

        QueryPerformanceFrequency(&f);
@@ -40,6 +45,16 @@
    LARGE_INTEGER t;
    QueryPerformanceCounter(&t);
    return static_cast<double>(t.QuadPart) / freq;
+ #else
+     // Windows reads the QueryPerformanceCounter clock GD stamps its input
+     // events with. On iOS the closest equivalent is the monotonic clock; if
+     // GD's own stamps use another time base, isLivePress() simply reports
+     // no live press and sub-tick recording records nothing -- it can never
+     // record a wrong time.
+     return std::chrono::duration<double>(
+         std::chrono::steady_clock::now().time_since_epoch())
+         .count();
+ #endif
    }

    bool isLivePress(PlayerButtonCommand const& cmd) {
diff --run a/tools/ios_audit.py b/tools/ios_audit.py
--- a/tools/ios_audit.py      1970-01-01 00:00:00.000000000 +0000
+++ b/tools/ios_audit.py      2026-10-05 11:22:56.699817176 +0000
@@ -0,0 +1,162 @@
+#!/usr/bin/env python3
+"""
+iOS "view" audit for GucciBot.
+
++There is no Geode SDK / Xcode in this sandbox, so the real compile cannot run
+here. This does the next best thing that CAN be checked offline:
+
++ 1. Rebuilds the exact source list CMake would compile for iOS.
++ 2. Preprocesses every file the way the iOS build sees it (GEODE_IS_WINDOWS
+   undefined, GB_NO_FFmpeg and GB_IOS defined), following only the conditional
+   branches that survive.
++ 3. Walks the #include graph from each iOS source (using the iOS include path:
+   src/ios/shims first, then src/) and reports any header that must never be
+   reached on iOS (FFmpeg, SafetyHook, Windows.h, Absense internals, ...).
++ 4. Scans the surviving lines for Windows-only constructs.
+
++Exit code 0 = clean, 1 = problems found.
+"""
+import re, sys, pathlib, collections
+
++ROOT = pathlib.Path(sys.argv[1]).resolve()
++SRC = ROOT / 'src'
++DEFINED = {'GB_NO_FFmpeg', 'GB_IOS', 'GEODE_IS_IOS', 'GEODE_IS_MOBILE', 'GEODE_I
++UNDEFINED ={'GEODE_IS_WINDOWS', '_WIN32', '_MSC_VER', 'GEODE_IS_X64', 'GEODE_IS
+
```

```

+# ---- 1. the iOS source list (mirrors CMakeLists.txt)
+def ios_sources():
+    out = []
+    for p in sorted(SRC.rglob('*.cpp')):
+        rel = p.relative_to(ROOT).as_posix()
+        if '/src/absense/' in '/' + rel or rel.startswith('src/absense/'): conti
+        if rel.startswith('src/mcp/'): continue
+        if re.match(r'src/render/(renderer|video_decoder|texture|pass)\.cpp$', r
+        out.append(p)
+    return out
+
+# ---- 2. tiny preprocessor
+def cond_value(expr):
+    e = expr.strip()
+    e = re.sub(r'//.*', '', e).strip()
+    m = re.fullmatch(r'defined\s*(?!\s*(\w+)\s*)?', e)
+    if m: return m.group(1) in DEFINED
+    m = re.fullmatch(r'!\s*defined\s*(?!\s*(\w+)\s*)?', e)
+    if m: return m.group(1) not in DEFINED
+    if e == '0': return False
+    if e == '1': return True
+    return None # unknown -> keep
+
+def ios_view(path):
+    """Return list of (lineno, text) that survive, and the include directives."""
+    lines = path.read_text(encoding='utf-8', errors='replace').splitlines()
+    stack = [] # each: [parent_active, this_branch_active, taken_any]
+    active = True
+    kept = []
+    for n, line in enumerate(lines, 1):
+        t = line.strip()
+        m = re.match(r'#\s*(ifdef|ifndef|if|elif|else|endif)\b(.*)', t)
+        if m:
+            kw, rest = m.group(1), re.sub(r'//.*|/\s*.*?*/', '', m.group(2))
+            if kw in ('ifdef', 'ifndef', 'if'):
+                if kw == 'ifdef': v = rest.strip() in DEFINED if rest.strip() in
+                elif kw == 'ifndef': v = (rest.strip() not in DEFINED) if rest.s
+                else: v = cond_value(rest)
+                if v is None: v = True
+                stack.append([active, v, v])
+                active = active and v
+            elif kw == 'elif':
+                par, cur, taken = stack[-1]
+                v = cond_value(rest)
+                if v is None: v = True
+                nv = (not taken) and v
+                stack[-1] = [par, nv, taken or nv]

```

```

+         active = par and nv
+     elif kw == 'else':
+         par, cur, taken = stack[-1]
+         nv = not taken
+         stack[-1] = [par, nv, True]
+         active = par and nv
+     elif kw == 'endif':
+         par, cur, taken = stack.pop()
+         active = par
+     continue
+     if active:
+         kept.append((n, line))
+     return kept
+
+INC = re.compile(r'#\s*include\s*([<"])([^\>"]+)[>"]')
+
+def resolve(inc, frm):
+    cand = [SRC / 'ios' / 'shims' / inc, frm.parent / inc, SRC / inc]
+    for c in cand:
+        if c.exists(): return c
+    return None
+
+## headers that must NOT be reached by the iOS build
+FORBIDDEN_REL = [
+    'render/ffmpeg.hpp', 'render/texture.hpp', 'render/pass.hpp', 'render/subpro
+    'render/ffmpeg_events.hpp', 'render/colospace/',
+]
+FORBIDDEN_ANGLE = ['safetyhook.hpp', 'windows.h', 'winsock2.h', 'ws2tcpip.h', 'z
+
+problems = []
+reached = collections.defaultdict(set)    # header -> {first including source}
+visited = set()
+
+def walk(path, origin):
+    if path in visited: return
+    visited.add(path)
+    for n, line in ios_view(path):
+        m = INC.search(line)
+        if not m: continue
+        kind, inc = m.group(1), m.group(2)
+        low = inc.lower()
+        if any(low.endswith(a) or low.startswith(a) or a in low for a in FORBIDD
+            problems.append(f'{path.relative_to(ROOT)}:{n}: includes forbidden "
+            continue
+        r = resolve(inc, path)
+        if r is None:
+            continue    # system / Geode / third-party header

```

```

+         rel = r.relative_to(SRC).as_posix() if SRC in r.parents else r.as_posix(
+         if any(rel.startswith(f) or rel == f for f in FORBIDDEN_REL):
+             problems.append(f'{path.relative_to(ROOT)}:{n}: reaches iOS-forbidde
+             continue
+         if rel.startswith('absense/') and not rel.startswith('absense/compat/')
+             problems.append(f'{path.relative_to(ROOT)}:{n}: reaches real Absense
+             continue
+         walk(r, origin)
+
+srcs = ios_sources()
+for s in srcs:
+    walk(s, s)
+
+# ---- 4. token scan on every surviving line of every file reached (+ the source
+TOKENS = [
+    (r'\bSafetyHook\w*|\bsafetyhook::', 'SafetyHook'),
+    (r'\bQueryPerformance\w+|\bLARGE_INTEGER\b', 'Win32 timer'),
+    (r'\bCreateProcess\w*|\bCreateFile\w*|\bHANDLE\b|\bDWORD\b|\bHMODULE\b|\bLoa
+    (r'\bWSA\w+|\bclosesocket\b|\bSOCKET\b|\bINVALID_SOCKET\b', 'Winsock'),
+    (r'base::get\(\)\s*\+\s*0x|getCocos\(\)\s*\+', 'raw address'),
+    (r'\bAV(Codec|Format|Frame|Packet|Stream|PixelFormat|Rational)\w*|\bSwsConte
+    (r'\b__declspec\b|\b__forceinline\b|\b_MSC_VER\b', 'MSVC-ism'),
+    (r'\bstrcpy_s\b', 'strcpy_s (shimmed in platform.hpp)'),
+]
+files = sorted(visited | set(srcs))
+scan_hits = []
+for f in files:
+    rel = f.relative_to(ROOT).as_posix()
+    for n, line in ios_view(f):
+        code = re.sub(r'//.*', '', line)
+        if not code.strip(): continue
+        for pat, name in TOKENS:
+            if re.search(pat, code):
+                if name.startswith('strcpy_s') and (rel.endswith('platform.hpp'))
+                scan_hits.append(f'{rel}:{n}: [{name}] {line.strip()[:110]}')
+
+print(f'iOS sources compiled      : {len(srcs)}')
+print(f'iOS files/headers reached  : {len(files)}')
+print(f'forbidden-include problems : {len(problems)}')
+print(f'Windows-only token hits    : {len(scan_hits)}')
+for p in problems: print('  PROBLEM ', p)
+for h in scan_hits[:80]: print('  TOKEN   ', h)
+sys.exit(1 if (problems or scan_hits) else 0)

```